



GoodScan

Technical Breakdown & the Supabase Data Layer

Everything behind the scenes: what is captured, how it travels, where it lands, and why.

```
repo      github.com/nikolassigmadev/greenproject
branch    fix/verdict-coherence    (clean, 438 commits)
runtime   https://goodscan.shop    |    com.scan2source.app
scope     47,700 LOC TypeScript    +    2,195-line Express server
```

CONTENTS

01	The product in one page	What GoodScan does and how a scan becomes a verdict
02	System architecture	Frontend, proxy backend, five persistence stores
03	The data layer map	Which store owns what, and why there are five
04	Supabase: connection & bootstrap	Pooler, TLS, self-applying schema, defensive init
05	ai_scans: the main table	All 20 columns, every source, every cap
06	The four write triggers	Exactly when a row appears in Supabase
07	The journey of one payload	Client -> HTTP -> sanitiser -> INSERT, end to end
08	The sanitisation gauntlet	Six functions that stand between users and the DB
09	unmet_ethical_demand	The view that turns logs into a business asset
10	community_flags	Crowdsourced flags, sourcing bar, moderation sync
11	What never reaches Supabase	The privacy boundary, enumerated
12	Operations & failure modes	Migration, reading data, resilience, risks

01

The product in one page

GoodScan is an **ethical shopping scanner**. A shopper points a phone at a grocery product; the app identifies it, cross-references it against a dozen hand-curated ethics datasets, and returns a single top-line verdict — **Buy, Consider, Caution, or Avoid** — weighted by the priorities that specific shopper declared during onboarding. The whole point of the app is the moment after the verdict: a **Buy / Skip decision bar**, and a **better swap** if we have one to offer.

That last sentence is what makes the data layer interesting. GoodScan does not just log *what people scanned*. It logs *what they decided*, *why the product was flagged*, and *whether we had an ethical alternative to offer them in their market*. That triple is the raw material for the only genuinely proprietary dataset the project produces.

The one-sentence version of the data strategy

Every scan writes a row that says: *a real shopper, in a real place, met a product with a real ethical problem — and here is whether the market could offer them a way out, and what they did about it*. Aggregate that and you have a live map of unmet ethical demand.

Pipeline, compressed:

```

camera photo / barcode / typed query
|
+-- 2D barcode?  gs1.ts pulls the GTIN out of a QR or GS1 DataMatrix
+-- 1D barcode?  barcodeValidator.ts (EAN-13/8, UPC-A, ISBN)
+-- photo?       OpenAI Vision ('scan-product') -> Product / Brand / Barcode
|
v
Open Food Facts lookup or search (proxied, never called from the browser)
|
+-- productRelevance.ts  distinctive vs. brand tokens, strong brand anchor
+-- visualMatch.ts       local colour histogram, then ONE AI call if unsure
|
v
getVerdict() + personalizedScore()  ->  BUY / CONSIDER / CAUTION / AVOID
|
v
DecisionBar: Buy or Skip      ->  THIS is what gets logged to Supabase

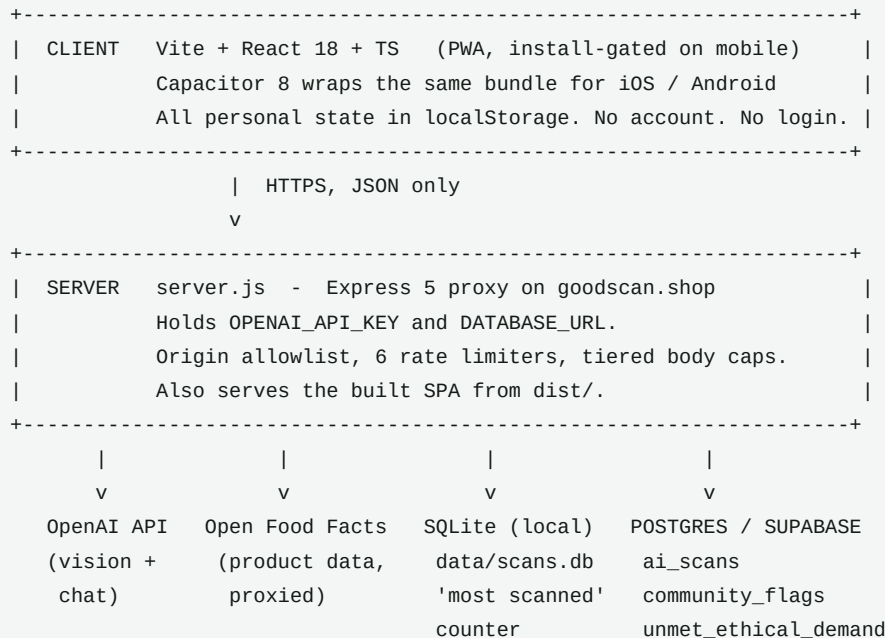
```

Scan-to-verdict latency was taken from roughly 13 seconds to 3.5–4.5 seconds by moving vision to gpt-4.1-mini, parallelising the Open Food Facts search strategies, time-boxing the visual match, and rendering the detail page before all enrichment resolves.

02

System architecture

Three tiers. The browser never holds a secret and never talks to a third party directly.



Why a proxy at all

- **Secrets.** The OpenAI key and the database URL exist only in the server process. A decompiled Android bundle yields nothing.
- **CORS.** Open Food Facts' Search-a-licious endpoint sends no CORS headers, so it is physically impossible to call from a browser. The proxy is the only route.
- **Prompt control.** All seven vision prompts and three chat prompts live server-side, so they can be tuned and redeployed without shipping a new app build.
- **Rate limiting and abuse control** sit in front of a metered paid API.
- **Observability.** The proxy is the natural chokepoint at which to log — which is exactly what the Supabase layer exploits.

Request-hardening at the edge

Control	Detail
Origin allowlist	Explicit list; unknown origins rejected before any handler runs.
trust proxy = 1	Exactly one reverse-proxy hop (Hostinger). Without this every visitor shares the proxy IP and one user could exhaust everyone's rate limit.
Six rate limiters	openai, search, auth, community-flag, scan (60/min), client-error — each scoped to its route group.
Tiered body caps	16 KB push / 4 KB small / 2 MB global / 4 MB scan / 10 MB image. A route gets the smallest parser that can serve it.
Admin sessions	bcrypt password check, in-memory token, 4-hour TTL, separate auth limiter.

03

The data layer map

Five distinct stores. Each exists because it answers a question the others cannot.

Store	Location	Holds	Why it is separate
localStorage	On device	Priorities, region, dietary rules, watchlist, basket, scan history, decisions, streaks	Privacy-first. This is the user's own data and it never has to leave the phone for the app to work fully offline.
SQLite	data/scans.db	barcode, name, brand, eco_grade, country, anon_id, ts	A fast, dependency-light 'most-scanned products' counter. Survives if Postgres is unreachable. Deliberately thin.
Postgres (Supabase)	Hosted, ap-northeast-1	ai_scans (rich per-scan log incl. photo + full model output), community_flags , unmet_ethical_demand view	The analytical store. Everything worth querying later, including the unmet-demand signal that is the product's strategic asset.
JSONL files	data/*.jsonl	openai-logs, community-flags, client-errors, push-subscriptions	On-disk backup and append-only audit trail. OpenAI purges its own dashboard logs after 30 days; this is the permanent record.
Bundle data	src/data/*.ts	35 brand flags, 55 chocolate entries, 26 verified-ethics brands, beef/egg/sugar/commodity datasets	The ethics knowledge base ships <i>with the app</i> , so verdicts render instantly and work with no network.

The important distinction

SQLite answers *"what do people scan?"*. Supabase answers *"what happened, to whom, where, why, and what did they do next?"* Only one of those is worth building a company on.

Note the deliberate asymmetry: a failed scan is written to Postgres but **not** to SQLite. Unresolved junk ("Unknown product", a stray typed query) would pollute the most-scanned leaderboard, so failures live only in the analytical store where they are debugging material rather than noise.

04

Supabase: connection & bootstrap

The entire Postgres integration lives in one 351-line module, `db/scanStore.js`, plus a single env var. There is no Supabase client library, no PostgREST, no row-level security, no Supabase Auth. It is plain pg over a connection string.

Connection string shape

```
DATABASE_URL=postgres://postgres.:
    @aws-1-ap-northeast-1.pooler.supabase.com:5432/postgres
```

Port 5432 on the pooler host is Supabase's **session pooler** — a full-featured session that supports prepared statements and everything the pg driver expects, unlike the transaction pooler on 6543. The credential currently sits in `.env.production`, which is gitignored.

Deliberately provider-agnostic

The module's own header states: *"Works against any Postgres connection string — Supabase, Neon, Railway, Aiven — via the single DATABASE_URL env var. No lock-in to any one provider."* Supabase is the current host, not a dependency. Swapping to Neon is a one-line env change.

Pool configuration

Setting	Value	Reasoning
max	5	Small ceiling — this is a logging workload, not a request path. Keeps well inside Supabase pooler limits.
idleTimeoutMillis	30,000	Releases idle connections back to the pooler.
connectionTimeoutMillis	8,000	Bounded so a network stall cannot pile up.
ssl	auto	TLS on unless the host is localhost/127.0.0.1. Override with DATABASE_SSL=false for local dev. rejectUnauthorized is false because Supabase's pooler presents a chain Node does not ship.
pool.on('error')	logged	Background connection drops are surfaced to the console, never thrown — an unhandled pool error would take the whole server down.

Self-applying schema

On startup, `initScanStore()` executes one multi-statement SQL blob against the database. It is written to be **fully idempotent** — `CREATE TABLE IF NOT EXISTS`, `ADD COLUMN IF NOT EXISTS`, `DROP COLUMN IF EXISTS`, `CREATE INDEX IF NOT EXISTS`, `CREATE OR REPLACE VIEW` — so a deploy against an old, a current, or an entirely empty database all converge to the same state with no migration tool, no version table, and no manual step.

It also actively *removes* five columns that were tried and abandoned: `carbon_footprint_100g`, `image_hash`, `image_url`, `model`, `query`, `ocr_text`. The schema garbage-collects itself.

Fails soft, always

If `DATABASE_URL` is unset, `init` logs a warning and returns. If the connection fails, it logs, sets `ready = false`, nulls the pool, and returns. Every write path is gated on `if (!ready || !pool) return;`. **The server runs perfectly with no database at all** — you simply lose analytics. This is the same defensive pattern used for the SQLite store, which can also fail to load its native module on a given host.

05

ai_scans — the main table

One row per meaningful scan event. Twenty columns. This is where the substance is.

Column	Type	What it actually contains	Cap / rule
id	BIGSERIAL	Primary key.	—
user_id	TEXT	A randomly generated UUID created on first use and kept in <code>localStorage</code> (<code>goodscan-anon-id</code>). Lets you count unique scanners and follow one device's journey. Not derived from any personal identifier.	64 chars
source	TEXT	'scan' or 'decision' — set server-side based on whether a buy/skip was attached.	64
product_name	TEXT	Resolved product name (post Open Food Facts match).	300
brand	TEXT	Resolved brand.	200
barcode	TEXT	EAN-13 / EAN-8 / UPC-A when one was read or resolved.	64
eco_grade	TEXT	Eco-Score letter A-E (or A-plus).	4
country	TEXT	Country code from the region the user set themselves in onboarding.	64
city	TEXT	Optional city, same explicit source.	120
off_url	TEXT	Derived server-side , not sent by the client: <code>world.openfoodfacts.org/product/{barcode}</code> . Makes every row one click from the source record.	derived
openai_response	TEXT	The <i>trimmed</i> string vision produced, e.g. "Cadbury Dairy Milk Caramel" — exactly what was handed to the Open Food Facts search.	500

Column	Type	What it actually contains	Cap / rule
full_openai_response	TEXT	The complete raw model output before trimming, e.g. the whole Product: / Brand: / Barcode: block. This is the column that makes mis-identifications forensically debuggable.	20,000
bought	TEXT	'YES' bought / 'NO' skipped / null no decision. The conversion signal.	enum
priorities	JSONB	Snapshot of the shopper's weights <i>at scan time</i> : {environment, laborRights, animalWelfare, nutrition}, each 0-100. Preserves why the verdict came out as it did even after they change settings.	0-100 ints
category	TEXT	Swap-catalog category, e.g. chocolate, coffee, eggs. One of 22.	64
verdict	TEXT	The verdict actually shown : BUY CONSIDER CAUTION AVOID UNKNOWN.	enum-checked
primary_concern	TEXT	Worst concern found: labor boycott animal_welfare eco. Null when clean.	enum-checked
swap_available	BOOLEAN	Was a region-available ethical alternative on offer? The single most commercially interesting boolean in the schema.	tri-state
image	TEXT	The actual photo the shopper took , inline as base64 JPEG (no data: prefix). 512 px, quality 0.72.	3M chars (~2.25 MB)
resolved	BOOLEAN	false = the scan never matched a product. These rows are the misses worth fixing.	default true
created_at	TIMESTAMP TZ	Server clock, defaults to now().	—

Why the photo is stored

Storing the scanned image inline is an unusual choice and a deliberate one. The image is already compressed client-side to 512 px at quality 0.72 — and that number is itself an optimisation finding: the backend requests OpenAI's detail: 'low' mode, which internally resizes to ~512 px and bills a **flat ~85 image tokens regardless of input size**. Shrinking further (the old 256 px at q0.4) cost OCR legibility and saved nothing. So the same compressed buffer that goes to the model is the one persisted. Together with full_openai_response and resolved = false, every failed identification can be reproduced exactly: *here is the photo, here is what the model said about it, here is why it did not resolve*.

Storage reality check

At the 3M-character ceiling a single row could be ~2.25 MB. A realistic 512 px q0.72 JPEG is 40–80 KB base64, so ~15,000 scans per GB. Supabase's free tier is 500 MB. **This column is the cost driver and the first thing that will need a lifecycle policy** — either offload to Supabase Storage and keep a URL, or drop images older than N days once the identification work is done.

06

The four write triggers

Rows do not appear on a timer. There are exactly four call sites, and each captures a different moment.

#	Trigger	Fires from	Row characteristics
1	Product page opened	OpenFoodFactsDetail.tsx:447	The main event. A product resolved and a verdict rendered. Carries brand, barcode, eco grade, verdict, priorities, and — via assessUnmetDemand() — category, primary concern and swap availability. When arrived from a camera scan, also carries the photo and both OpenAI response fields (pulled from sessionStorage). bought = null.
2	Buy / Skip pressed	DecisionBar.tsx:89	A second row for the same product, this time with bought = 'YES' 'NO' and source = 'decision'. The demand signals are re-assessed at decision time. This is the conversion event.

#	Trigger	Fires from	Row characteristics
3	Camera scan failed	Scan.tsx:1331 BarcodeScannerOverlay.tsx:426,432	resolved = false, verdict = 'UNKNOWN', plus the photo and the raw model output. Never written to SQLite. Pure debugging corpus.
4	Text search found nothing	Scan.tsx:1745 BarcodeScannerOverlay.tsx:474	resolved = false with the raw typed text, and the AI-cleaned form recorded alongside as typed: "x" -> cleaned: "y". Shows what people look for that the database cannot serve.

The two-row pattern is intentional

A shopper who scans and then skips produces two rows: one at source = 'scan' and one at source = 'decision'. This is not duplication — it separates *exposure* from *conversion*. You can compute a skip rate per brand, per verdict, per concern type, and measure whether showing an AVOID verdict actually changes behaviour. Collapsing them into one row would destroy that.

Client-side guarantees on every trigger

- **Fire-and-forget.** `void fetch(...)` with a `.catch(() => {})`. Logging can never block or break a scan.
- **4-second timeout** via `AbortSignal.timeout(4000)`.
- **Conditional keepalive.** `keepalive: body.length < 60_000` — keepalive lets a log survive page navigation, but browsers *reject* keepalive bodies over 64 KB outright. A photo-bearing payload blows past that, so image logs must go as a normal fetch or they silently never leave the device. This is a real bug that was found and fixed.
- **Opt-out honoured.** `goodscan-scan-logging-optout` in `localStorage` short-circuits the whole function. (No UI exposes it yet — worth adding.)
- **Whole body wrapped in try/catch.** The comment reads: *"never let logging break a scan"*.

07

The journey of one payload

End to end, for a single scan of a flagged chocolate bar in the UK.

STEP 1 -- CLIENT assembles the body (src/utils/scanLogger.ts)

```

region      = loadRegion()           // explicit, user-set. NEVER IP-derived.
priorities  = loadPriorities()       // current weights snapshot
demand      = assessUnmetDemand(product, priorities, region.countryCode)

POST https://goodscan.shop/api/scans
{
  name: 'Dairy Milk Caramel', brand: 'Cadbury', barcode: '7622210449283',
  ecoGrade: 'd', verdict: 'AVOID', bought: 'NO',
  country: 'GB', city: 'Manchester', anonId: '9f2c...-uuid',
  priorities: { environment: 50, laborRights: 100, animalWelfare: 50, nutrition: 50 },
  category: 'chocolate', primaryConcern: 'labor', swapAvailable: true,
  openaiResponse: 'Cadbury Dairy Milk Caramel',
  fullOpenaiResponse: 'Product: Dairy Milk Caramel\nBrand: Cadbury\nBarcode: 762221...',
  image: '/9j/4AAQSkZJRgABA... // 512px q0.72 JPEG base64
  resolved: true
}
```

STEP 2 -- SERVER receives it (server.js -> POST /api/scans)

```

scanLimiter  60 requests / minute / IP
scanBody     express.json({ limit: '4mb' }) // sized for the photo
guard       if (!name) -> 400
```

```

didResolve = resolved !== false;

if (didResolve) recordScan({...}) // SQLite counter -- failures excluded

logScan({
  // Postgres -- everything, incl. failures
  source: bought ? 'decision' : 'scan',
  userId: anonId, productName: name, ...
});

res.json({ success: true }); // returns BEFORE the INSERT completes

```

```

STEP 3 -- SANITISER + INSERT (db/scanStore.js -> logScan)

if (!ready || !pool) return; // no DB? silently skip

barcode = clip(rec.barcode, 64)
offUrl = barcode ? `https://world.openfoodfacts.org/product/${barcode}` : null
bought = (rec.bought === 'YES' || rec.bought === 'NO') ? rec.bought : null
verdict = oneOf(rec.verdict, VERDICTS) // else null
concern = oneOf(rec.primaryConcern, CONCERNS) // else null
prio = priorityJson(rec.priorities) // 4 numeric keys ONLY
img = imageData(rec.image) // strip prefix, cap 3M

pool.query('INSERT INTO ai_scans (...19 cols...) VALUES ($1..$19)', values)
  .catch(e => console.error('scanStore: insert failed --', e.message));

// NOT awaited. A DB hiccup logs to console and is swallowed.

```

Note what is NOT in the payload

No name, no email, no IP, no precise coordinates, no device fingerprint, no advertising ID, no session cookie. The strongest identifier on the row is a random UUID the user's own browser minted and could clear at any time.

08

The sanitisation gauntlet

Six pure functions stand between arbitrary client JSON and the database. Every single column passes through one of them.

Function	Behaviour	Protects against
<code>clip(s, n)</code>	Collapses all whitespace and control runs to single spaces, trims, truncates to n. Returns null for empty.	Unbounded strings, control-character injection, log-line forgery, ragged data.
<code>clipRaw(s, n)</code>	Same but preserves newlines : normalises CRLF to LF, collapses spaces/tabs only, caps blank-line runs at two, then truncates.	Used only for <code>full_openai_response</code> , where multi-line structure is the point and must stay legible.
<code>oneOf(s, set)</code>	Clips to 32 chars, then returns the value only if it is in an allowlist , else null.	Junk in the heatmap dimensions. Guards verdict (5 values) and primary_concern (4 values). An attacker cannot invent a new verdict category.
<code>num(v, max)</code>	Coerces, then requires finite, ≥ 0 , $\leq$ max.	NaN, Infinity, negatives, and absurd magnitudes reaching JSONB.

Function	Behaviour	Protects against
priorityJson(p)	Iterates a fixed key list (environment, laborRights, animalWelfare, nutrition), runs each through num(v, 100), rounds, and rebuilds a fresh object. Unknown keys are dropped, not copied .	The module comment is explicit: <i>"Never stores free-text — no PII can leak in here."</i> A client cannot smuggle arbitrary JSON into the JSONB column by adding keys.
imageData(s)	Strips any data: URI prefix by slicing after the first comma, trims, and rejects anything over 3,000,000 characters.	Payload bloat and row explosion. Note it is a length check, not a format validation — see risks.

Structural safety

- **Every write is a parameterised query.** All 19 values go in as \$1 . . \$19 bindings. There is no string concatenation into SQL anywhere in the module. SQL injection is structurally impossible on this path.
- **JSONB is cast explicitly** as \$13::jsonb from a JSON.stringify of a freshly built object.
- **Enum columns are validated in application code**, not by a Postgres CHECK constraint — a pragmatic choice that keeps the self-applying schema simple, at the cost of the DB not enforcing it independently.
- **Nothing throws.** The outer try/catch plus the promise .catch() mean a malformed record produces a console line, not a 500.

09

unmet_ethical_demand — the payoff

This view is the reason the schema has the shape it has. Everything else is instrumentation; this is the asset.

```
CREATE OR REPLACE VIEW unmet_ethical_demand AS
SELECT
  country,
  city,
  category,
  primary_concern,
  count(*)                               AS demand_signals,
  count(*) FILTER (WHERE bought = 'NO')  AS rejected,
  count(DISTINCT user_id)                AS distinct_users,
  max(created_at)                        AS last_seen
FROM ai_scans
WHERE category IS NOT NULL
  AND primary_concern IS NOT NULL  -- the product carried a real ethical concern
  AND swap_available IS NOT TRUE   -- ...and we had NO alternative to offer
GROUP BY country, city, category, primary_concern
ORDER BY rejected DESC, demand_signals DESC;
```

How to read a row

GB | Manchester | chocolate | labor | 412 | 158 | 96 | 2026-08-14

In Manchester, 96 distinct shoppers hit chocolate carrying labour concerns 412 times, and on 158 of those occasions they walked away — and we could not offer them a single ethical alternative actually sold in the UK.

Metric	Meaning	Commercial reading
demand_signals	Every encounter with a flagged product where no swap existed.	Total addressable frustration. Market size for this gap.

Metric	Meaning	Commercial reading
rejected	The subset where the shopper actually pressed Skip .	Acute unmet demand. They wanted out and had nowhere to go. This is a lost sale someone could capture. The view sorts by this first, deliberately.
distinct_users	Unique devices behind the signals.	Separates a genuine market from one obsessive tester.
last_seen	Most recent occurrence.	Is this gap live or historical?

Why this is defensible data

Nobody else has it. Retail sales data tells you what people **bought**. Survey data tells you what people **say** they care about. This tells you what people **refused to buy, on ethical grounds, at the shelf, in a named city, when no alternative existed** — with the concern type attached. It is a direct sourcing and stocking brief: which product, which market, which ethical axis, ranked by how many shoppers already walked away.

The supporting partial index

```
CREATE INDEX IF NOT EXISTS idx_ai_scans_demand
  ON ai_scans (country, category, primary_concern)
  WHERE primary_concern IS NOT NULL AND swap_available IS NOT TRUE;
```

A **partial** index whose predicate mirrors the view's WHERE clause exactly. It indexes only the rows the heatmap cares about — likely a small fraction of the table — which keeps it cheap to maintain on a write-heavy log while making the aggregation fast.

Where swap_available comes from

assessUnmetDemand() in src/services/swaps/index.ts diagnoses the product's worst concern, maps it to one of 22 catalog categories, then pools candidates from four sources (curated alternatives, verified-ethics brands, the chocolate directory, custom swaps) and asks whether **any** candidate simultaneously: passes isCandidateClean() (not boycotted, no labour allegations, no critical/high welfare flag, no verified labour flag), addresses *this specific* concern type, has verifiable availability, and is sold in the user's country. Only then is swap_available = true. The bar is deliberately high, so a false is meaningful.

10

community_flags — crowdsourced evidence

The second Supabase table. Users submit brand allegations at /submit-flag; the server validates hard, writes to JSONL as the on-disk backup, and mirrors to Postgres.

Column	Contents
id	Generated cf_{timestamp}_{6 random bytes hex}. Primary key.
status	pending_review approved rejected. Defaults to pending_review.
brand_name	2-80 characters, required.
category	One of ten: forced_labour, child_labour, wage_theft, unsafe_conditions, union_busting, discrimination, supply_chain_opacity, animal_welfare, environmental_harm, boycott_listed.
severity	critical high medium low.
summary	10-300 characters. Rejected if all-caps (6+ letters, all uppercase) — a neat low-tech rant filter.
sources	JSONB array of 1-5 entries, each requiring a valid http(s) url , a title , a publisher , and a tier of tier1/tier2/tier3.
submitter_email	Optional, max 200 chars. The only optional PII field in the system.

Column	Contents
meets_sourcing_bar	Computed server-side , not submitted. See below.
ip_hash	SHA-256 of (client IP + daily UTC salt) . The raw IP is never persisted. The salt rotates at UTC midnight so hashes cannot be correlated across days.
moderator_note	Up to 500 chars, set on approve/reject.
submitted_at / moderated_at	TIMESTAMPTZ.

The sourcing bar

A submission is scored against the identical rule the in-house flag schema uses, so community evidence and curated evidence are held to one standard:

`meetsSourcingBar(sources)` is TRUE when any of:

- | | |
|--|-------------------------------|
| (1) 1 or more tier-1 sources | -- e.g. US DOL, court filing |
| (2) 2 or more tier-2 sources from DIFFERENT publishers | -- corroboration, not an echo |
| (3) 1 tier-2 source + 2 or more tier-3 sources | |

Condition (2) is the sharp one: it dedupes on lowercased publisher name, so two articles syndicated from the same outlet do not count as corroboration.

Anti-abuse

- **Honeypot field.** The client posts a realistically-named `company_website` field that legitimate users leave empty. Bots fill every field. On a hit the server returns **HTTP 200 with a fake id** (`cf_ignored`) so spammers never learn the trap exists.
- **Dedicated rate limiter** separate from the general API limiters.
- **4 KB body cap** — no room for a payload attack.
- **Strict field-by-field validation** with indexed error messages (`sources[2].tier must be...`).

Moderation sync

PATCH `/api/admin/community-flags/:id` rewrites the JSONL record *and* calls `updateCommunityFlagStatus()` to reflect the decision onto the Postgres row. The insert uses `ON CONFLICT (id) DO NOTHING`, so replaying the JSONL is safe.

Dual-write, JSONL as the source of truth

The admin *read* endpoint reads the JSONL file, not Postgres. Postgres is the queryable mirror. That means if a Postgres write fails silently, the two can drift — the JSONL stays correct and the database quietly misses a flag. A periodic reconciliation job (replaying the JSONL through the idempotent insert) would close this.

11

What never reaches Supabase

The privacy posture is not incidental — it is enforced in several independent places.

Never transmitted	Where it stays	Enforced by
Scan history (the user's own timeline)	localStorage	userPreferences.ts — read/written on-device only
Basket contents	localStorage	basketStorage.ts
Watchlist (brands trusted / avoided)	localStorage	watchlist.ts
Dietary preferences and allergens	localStorage	dietaryPreferences.ts. Health data — deliberately never leaves the device.
Buy/skip decision <i>history</i>	localStorage	decisions.ts. Only the single boolean outcome is logged, never the log itself.
Streaks, swap tracking, weekly recap	localStorage	Derived on-device from the above.
Any IP-derived location	Nowhere	Explicit project rule. Country and city come only from what the user typed in onboarding. There is no geo-IP call anywhere in the codebase.
Raw client IP (community flags)	Never persisted	hashIp() — SHA-256 with a daily-rotating UTC salt
OpenAI API key, DATABASE_URL	Server process only	Proxy architecture; client bundles contain neither

The identity model

There are no accounts. No login, no email, no OAuth, no Supabase Auth. Identity is a single random UUID minted by `crypto.randomUUID()` on first use and stored under `goodscan-anon-id`. It is not derived from hardware, network, or anything about the person. Clearing site data mints a new one, and the old rows become permanently unlinkable.

Honest framing

This is genuinely privacy-respecting for an analytics pipeline, but it is not anonymity. The combination of a stable device UUID, a self-declared city, timestamps, and a photograph of a product held in someone's hand is re-identifiable in principle. Two things follow: (a) the **image** column is the highest-sensitivity field in the system and deserves a retention limit; (b) the `goodscan-scan-logging-optout` flag is honoured but has **no UI** — surfacing it in Preferences would make the privacy story defensible end to end.

12

Operations, failure modes & open risks

Getting data out

Path	Reaches	Notes
Supabase SQL editor	Postgres directly	The primary analytical route. <code>db/schema.sql</code> ships example queries: all scans for a product, one user's history, most-scanned, and <code>SELECT * FROM unmet_ethical_demand LIMIT 50</code> .
<code>scripts/pull-scans.sh</code>	SQLite only	Prompts for the admin password with echo disabled, logs in, downloads <code>/api/admin/scans</code> to <code>~/Downloads</code> , logs out, then pretty-prints a top-25 with <code>jq</code> . Note: this hits the SQLite counter, not Postgres.
<code>scripts/view-scans.js</code>	SQLite only	Local reader for a <code>data/scans.db</code> file. CSV export.
<code>scripts/pull-flags.sh</code>	JSONL	Admin-gated community-flag download.
<code>scripts/migrate-ai-scans.mjs</code>	Postgres	Optional, idempotent, additive-only migration. Reads <code>DATABASE_URL</code> from <code>env</code> , then <code>.env.local</code> , then <code>.env.production</code> . The server applies the same schema on boot, so this only saves a restart.

Failure modes and how each is handled

Failure	Behaviour
DATABASE_URL unset	Warning at boot. All Postgres writes no-op. App fully functional.
Supabase unreachable at boot	init catches, ready=false, pool nulled. Writes no-op.
Connection drops later	pool.on('error') logs. Next query fails, .catch() swallows it.
INSERT fails (constraint, size, timeout)	console.error, row lost. No retry, no dead-letter queue. Accepted trade-off for analytics.
better-sqlite3 native module won't load	Warning; SQLite disabled; Postgres continues.
Both stores unavailable	Only then does /api/scans return 503 . Otherwise it always returns 200 without waiting for the insert.
Client offline / server down	fetch .catch() no-ops. The scan itself is unaffected.

Open risks, ranked

1. Credentials — act on this first

.env.production holds a live-format OpenAI key and a Supabase connection string with a **plaintext password**. The *current* key is not in git history and HEAD is clean — but an **older key was committed**, in ADVANCED_OCR_DOCUMENTATION.md and a bundled Android JS asset (commits 8f814c4, 3cad033, a3844c6), on a **public** remote. Those files were deleted later, but git history retains them. Rotation of both the key and the Supabase password was recommended and is still unconfirmed. Separately, ADMIN_PASSWORD_HASH is still the literal placeholder \$2b\$10\$REPLACE_THIS_WITH_A_REAL_HASH, so admin auth silently falls back to the hash hardcoded in server.js.

2. No row-level security, no service-role separation

The app connects as the postgres superuser role. Supabase RLS is not enabled on either table. This is defensible while the *only* client is a trusted server process — but it means that connection string is equivalent to full database access, which raises the stakes on point 1 considerably.

3. The image column will become the cost centre

No retention policy, no lifecycle rule, no offload to object storage. Rows are append-only forever. Recommended: keep images for unresolved scans (where they earn their keep) and expire them for resolved ones after ~30 days.

4. Smaller items

(a) Enum validation lives in app code, not DB constraints — the database will accept anything a future writer sends. **(b)** The JSONL/Postgres dual-write for community flags can drift with no reconciliation. **(c)** imageData() checks length but not that the payload is actually valid base64 JPEG. **(d)** The opt-out flag has no UI. **(e)** pull-scans.sh is named as though it reads the rich log but only reaches SQLite — there is no admin HTTP route onto Postgres at all.

Verified at time of writing: 238 tests passing, 35 skipped (live E2E, gated behind RUN_LIVE=1), 19 test files green. Working tree clean on fix/verdict-coherence. CI runs typecheck, eslint, vitest and build on every push and PR.